

Aplicaciones reales de las funciones hash criptográficas

José de Jesús Angel Angel

Facultad de Ingeniería y Ciencias Aplicadas, Universidad Anáhuac México

20 de agosto de 2026

Resumen

Las funciones hash criptográficas —funciones deterministas de un solo sentido, resistentes a colisiones y a preimágenes— constituyen uno de los componentes más ubicuos de la criptografía moderna. Este documento presenta un panorama de sus aplicaciones reales en sistemas desplegados actualmente, con referencias a los estándares y publicaciones técnicas correspondientes.

1 Introducción

Una función hash criptográfica $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ debe satisfacer, informalmente, tres propiedades: resistencia a preimagen, resistencia a segunda preimagen y resistencia a colisiones. Familias como SHA-2 y SHA-3, estandarizadas por el NIST, cumplen estas propiedades bajo los modelos de seguridad actuales [1, 2]. A continuación se listan aplicaciones concretas en las que estas funciones son un componente crítico de sistemas reales.

2 Lista de aplicaciones

1. Verificación de integridad de archivos

Los *checksums* (por ejemplo, `sha256sum`) permiten detectar si un archivo descargado ha sido corrompido o alterado, comparando el hash publicado por el distribuidor contra el hash calculado localmente. Es el mecanismo estándar de distribución de software (kernels de Linux, imágenes ISO, paquetes de `pip/npm`) [1].

2. Firmas digitales (esquema hash-then-sign)

Los algoritmos de firma digital como RSA-PSS y ECDSA no firman el mensaje completo, sino su hash. Esto reduce el costo computacional y evita ataques de maleabilidad. El estándar del NIST para firmas digitales especifica explícitamente el uso de SHA-2/SHA-3 como paso previo a la firma [3, 4].

3. Almacenamiento seguro de contraseñas

En lugar de guardar contraseñas en claro, los sistemas almacenan $H(\text{contraseña} \parallel \text{sal})$ usando funciones de derivación de clave diseñadas para ser computacionalmente costosas, como PBKDF2, `bcrypt` o `Argon2`, mitigando ataques de fuerza bruta y de diccionario [5, 6].

4. Autenticación de mensajes (HMAC)

HMAC combina una función hash con una clave secreta para producir un código de autenticación de mensajes (MAC) que garantiza integridad y autenticidad. Se usa en APIs REST, tokens JWT (algoritmo HS256) y en protocolos de red [7].

5. Protocolos TLS/SSL

TLS 1.3 utiliza funciones hash (SHA-256/SHA-384) dentro de HKDF para derivar las claves de sesión, y dentro de HMAC para autenticar los mensajes del *handshake*. Todo tráfico HTTPS depende de esta maquinaria [8, 9].

6. Blockchain y prueba de trabajo (Bitcoin)

Bitcoin utiliza SHA-256 aplicado dos veces (*double-SHA256*) tanto para encadenar bloques como para el algoritmo de *proof-of-work*, en el cual los mineros buscan un nonce tal que el hash del encabezado del bloque quede por debajo de un umbral de dificultad [10].

7. Árboles de Merkle

Los árboles de Merkle permiten verificar la pertenencia de un dato a un conjunto grande mediante una prueba logarítmica en el tamaño del conjunto. Se usan en Bitcoin y Ethereum para resumir las transacciones de un bloque, y en sistemas de *certificate transparency* y bases de datos distribuidas [11].

8. Control de versiones (Git)

Git identifica cada objeto (commit, árbol, blob) mediante el hash de su contenido, lo que garantiza integridad del historial completo del repositorio; un cambio en cualquier commit altera todos los hashes posteriores. Git migró de SHA-1 a SHA-256 tras la demostración práctica de colisiones (*SHAttered*) [12, 13].

9. Firma de código y verificación de software

Sistemas como Authenticode (Microsoft) y *codesign* (Apple) hashean el binario ejecutable y firman ese hash, permitiendo al sistema operativo verificar que el software no fue modificado desde su publicación por el desarrollador [3].

10. Transparencia de certificados (Certificate Transparency)

El estándar RFC 6962 usa árboles de Merkle sobre certificados X.509 para crear registros públicos, append-only y auditables, permitiendo detectar certificados TLS fraudulentos emitidos por autoridades certificadoras comprometidas [14].

11. Deduplicación y almacenamiento direccionado por contenido

Sistemas de archivos y de respaldo (IPFS, Git, ZFS, Dropbox) usan el hash del contenido como identificador único, evitando almacenar copias duplicadas y facilitando la verificación de integridad distribuida [15].

12. Cómputo forense y listas de referencia (NSRL)

El *National Software Reference Library* del NIST mantiene bases de datos de hashes de archivos conocidos, usadas por investigadores forenses para descartar rápidamente archivos de sistema legítimos en una investigación digital [16].

3 Conclusión

Las funciones hash criptográficas son un componente estructural —no accesorio— de la seguridad informática contemporánea: subyacen a la web segura (TLS), a las criptomonedas, al control de versiones de casi todo el software moderno, y a la autenticación de billones de usuarios diariamente. Su análisis formal, sin embargo, sigue siendo un área activa de investigación, particularmente en torno a la resistencia cuántica de las construcciones actuales [2].

Referencias

- [1] NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [2] NIST, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS PUB 202, 2015. <https://doi.org/10.6028/NIST.FIPS.202>
- [3] NIST, *Digital Signature Standard (DSS)*, FIPS PUB 186-5, 2023. <https://doi.org/10.6028/NIST.FIPS.186-5>
- [4] K. Moriarty, B. Kaliski, J. Jonsson, A. Rusch, *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017, IETF, 2016. <https://doi.org/10.17487/RFC8017>
- [5] B. Kaliski, *PKCS #5: Password-Based Cryptography Specification Version 2.0*, RFC 2898, IETF, 2000. <https://doi.org/10.17487/RFC2898>
- [6] NIST, *Digital Identity Guidelines: Authentication and Lifecycle Management*, SP 800-63B, 2017. <https://doi.org/10.6028/NIST.SP.800-63b>
- [7] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, 1997. <https://doi.org/10.17487/RFC2104>
- [8] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, 2018. <https://doi.org/10.17487/RFC8446>
- [9] H. Krawczyk, P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, IETF, 2010. <https://doi.org/10.17487/RFC5869>
- [10] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008. <https://bitcoin.org/bitcoin.pdf>
- [11] R. C. Merkle, *A Digital Signature Based on a Conventional Encryption Function*, en *Advances in Cryptology — CRYPTO '87*, LNCS 293, pp. 369–378, Springer, 1988. https://doi.org/10.1007/3-540-48184-2_32
- [12] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, Y. Markov, *The First Collision for Full SHA-1*, en *Advances in Cryptology — CRYPTO 2017*, LNCS 10401, pp. 570–596, Springer, 2017. https://doi.org/10.1007/978-3-319-63688-7_19

- [13] Git Project, *Git: Hash Function Transition*, 2024. <https://git-scm.com/docs/hash-function-transition>
- [14] B. Laurie, A. Langley, E. Kasper, *Certificate Transparency*, RFC 6962, IETF, 2013. <https://doi.org/10.17487/RFC6962>
- [15] J. Benet, *IPFS – Content Addressed, Versioned, P2P File System*, 2014. <https://arxiv.org/abs/1407.3561>
- [16] NIST, *National Software Reference Library (NSRL)*, 2024. <https://www.nist.gov/itl/ssd/software-quality-group/national-software-reference-library-nsrl>